



SQL

*Structured Query
Language*

Dettaglio della lezione

In questa lezione vedremo cos'è SQL e la sua terminologia, e come operare sul data base attraverso questo linguaggio.

Nello specifico vedremo:

- come creare, modificare e cancellare tabelle
- come interrogare i dati memorizzati nel nostro data base
- come lavorare su più tabelle contemporaneamente
- le query di aggiornamento e di estrazione

Cos'è SQL?

L'acronimo SQL sta per Structured Query Language e rappresenta il linguaggio attraverso il quale è possibile interagire con i database.

SQL, quindi, fornisce un modo per estrarre e manipolare i dati memorizzati in un database relazionale, comprese le operazioni di definizione dei dati.

SQL comunque non deve essere confuso con un linguaggio di programmazione. Questo infatti non fornisce alcuna funzione di controllo del flusso dei dati, come funzioni, cicli o strutture if-else.

Note sul SQL

SQL non è case sensitive, il che vuol dire che i comandi si possono scrivere indifferentemente sia in minuscolo che in maiuscolo.

Bisogna comunque fare attenzione ai contenuti da ricercare: in quel caso le ricerche, se non specificato diversamente, terranno conto dei caratteri da ricercare così come specificati.

SQL come DDL e DML

SQL viene spesso definito come “linguaggio composto”; questo si suddivide infatti in:

- **DDL** (*Data Definition Language*)
- **DML** (*Data Manipulation Language*)

Il DDL si occupa e comprende tutti quei comandi finalizzati alla definizione dei dati, come creazione e gestione dei database e delle tabelle.

Il DML, invece, comprende tutti quei comandi finalizzati alla manipolazione dei dati come le operazioni di ricerca o di aggiornamento dei dati.



DDL

Data Definition Language

Comandi per la definizione di schemi

Per schema si intende una singola base di dati.

Con SQL si possono creare, cancellare e gestire i data base presenti.

Di seguito l'elenco dei comandi appartenenti al DDL (data definition language).

Create database

Il comando CREATE DATABASE <nome> crea un data base con il nome specificato:

//Creo un data base di nome studenti

CREATE DATABASE studenti;

// e un altro di nome libreria

CREATE DATABASE libreria;

Drop database

Il comando DROP DATABASE<nome> elimina lo schema specificato e tutte le tabelle in esso contenute:

//Elimino il data base di nome sito e tutto il suo contenuto

DROP DATABASE sito;

//Elimino il data base di nome dbProva e tutto il suo contenuto

DROP DATABASE dbProva;

Comandi per la definizione e gestione delle tabelle

Come già visto, un data base è costituito da tabelle.

Con SQL si possono creare, cancellare e gestire le varie tabelle.

Di seguito l'elenco dei comandi seguiti da un semplice esempio.

Create table

Il comando CREATE TABLE <nome> (<definizione>) crea una tabella con il nome specificato e con la definizione della struttura indicata tra parentesi, dove separati da virgola vengono specificati nome e tipo dei campi.

//Creo una tabella di nome negozio con campi id, nome, citta, cap rispettivamente di tipo numerico, alfanumerico di 25 caratteri, alfanumerico di 25 caratteri, alfanumerico di 80 caratteri, alfanumerico di 5 caratteri

create table negozio (id NUMERIC, nome VARCHAR(25), citta VARCHAR(25), indirizzo VARCHAR(80), cap VARCHAR(5));

Create table è un comando tutto sommato semplice. Ovviamente è necessario avere idee ben precise di come deve essere la struttura della tabella che si vuole creare e dei campi che questa conterrà.

Oltre alla tabella e ai suoi campi bisognerà fare attenzione anche ai tipi di dato specificati.

Informazioni riguardanti la definizione di un data base e della sua architettura sono presenti nelle lezioni della stessa unità didattica.

Per maggiori informazioni riguardo i tipi di dato vi invitiamo a leggere l'appendice apposita.

Rename

Il comando

RENAME <vecchio nome tabella> TO <nuovo nome>

rinomina la tabella specificata.

//Rinomino la tabella di nome copia_negozio in negozio

rename copia_negozio to negozio;

Drop table

Il comando DROP TABLE <nome> elimina la tabella con il nome specificato ed il suo eventuale contenuto.

Attenzione a questo comando: drop table elimina fisicamente la tabella specificata, il che non corrisponde ad eliminare il solo contenuto (il cui comando lo vedremo più avanti nella guida), che lascerebbe invece definita la tabella stessa e la sua struttura.

//Elimino la tabella di nome negozio

drop table negozio;

Alter table

Il comando ALTER TABLE <nome> <comando secondario> modifica la tabella con il nome specificato applicando su di essa il comando indicato.

Con alter table è quindi possibile eseguire una modifica alla struttura della tabella.

E' quindi possibile modificare colonne, campi, chiavi, e nomi.

A seguire un esempio dei principali utilizzi del comando alter table.

Trattandosi di tante possibili varianti, si consiglia fortemente di consultare la documentazione ufficiale SQL per maggiori e più dettagliate informazioni.

// Il comando che segue elimina la chiave primaria, precedentemente specificata per la tabella indicata

ALTER TABLE <nome tabella> DROP PRIMARY KEY;

ALTER TABLE libri DROP PRIMARY KEY;

// Il comando che segue aggiunge una chiave primaria alla tabella. Il campo specificato tra parentesi sarà la nuova chiave primaria. E' possibile indicare più campi separandoli da virgola (,)

ALTER TABLE <nome tabella> ADD PRIMARY KEY (<campo chiave>);

ALTER TABLE libri ADD PRIMARY KEY (CodISBN);

// Il comando che segue aggiunge la colonna specificata

ALTER TABLE <nome tabella> ADD <nome colonna> <tipologia>;

ALTER TABLE libri ADD descrizioneLibro varchar(60);

// Il comando che segue modifica la struttura della colonna specificata. E' possibile ad esempio andare a modificare gli attributi della colonna descrizioneLibro precedentemente creata per rendere il campo più capiente. E' inoltre possibile modificare più colonne contemporaneamente separando i valori da virgola

ALTER TABLE <nome tabella> MODIFY <nome colonna> <attributi>, <nome colonna> <attributi>;

ALTER TABLE libri MODIFY descrizioneLibro varchar(80);

// Il comando che segue elimina la colonna specificata. E' possibile ad esempio andare a modificare gli attributi della colonna descrizioneLibro precedentemente creata per rendere il campo più capiente. E' inoltre possibile modificare più colonne contemporaneamente separando i valori da virgola

ALTER TABLE <nome tabella> DROP COLUMN <nome colonna>;

ALTER TABLE libri DROP COLUMN descrizioneLibro;



DML

Data Manipulation Language

Comandi per la manipolazione dei dati

Il DML comprende tutti quei comandi di manipolazione dei dati come le operazioni di ricerca o di aggiornamento dei dati. Le operazioni di manipolazione vengono comunemente chiamate query (interrogazione) e si possono suddividere in:

- query di estrazione;
- query di aggiornamento.

Query di estrazione

Le query di estrazione sono identificate con l'istruzione `SELECT`, che è probabilmente l'istruzione più utilizzata dell'sql. Le `select` permettono di interrogare il nostro data base e di ottenere come risultato tutti i record che soddisfano i nostri criteri di ricerca.

Il risultato ottenuto da una query è a tutti gli effetti una tabella, composta quindi da righe e colonne.



Le colonne saranno i campi che specificheremo nella select, mentre le righe rappresentano ogni record con le caratteristiche da noi indicate.

L'istruzione select può essere molto complessa e può essere composta da più clausole che determinano la complessità dell'interrogazione da effettuare.

A seguire vedremo la struttura dell'istruzione e dei suoi esempi.

Query di aggiornamento

Le query di aggiornamento sono identificate con le istruzioni INSERT INTO, UPDATE e DELETE e ci permettono di eseguire sul nostro data base operazioni di inserimento, modifica o cancellazione dei dati.

A seguire vedremo la struttura di tali istruzioni e degli esempi.

L'istruzione SELECT

L'istruzione SELECT

Come detto, la SELECT è l'istruzione più utilizzata nell'SQL avendo lo scopo di ricercare e quindi estrarre dei dati da una o più tabelle.

Nella select è inoltre possibile specificare i criteri di selezione, di raggruppamento e di ordinamento.

Vediamo quindi la sintassi dell'istruzione e le varie clausole che la compongono.

Sintassi dell'istruzione SELECT

SELECT [operatori] <elenco campi>

FROM <tabella>

[JOIN <tabella> ON <campo di unione>]

[WHERE <criteri di selezione>]

[GROUP BY <campi >]

[HAVING <criteri di selezione>]

[ORDER BY <criteri di ordinamento>]

Dentro le parentesi quadre le clausole facoltative. Se ne deduce quindi che l'istruzione più semplice e obbligatoria per estrarre dei dati comincerà sempre con SELECT [operatori] <elenco campi> FROM<tabella>

Partendo dalla tabella sotto riportata vediamo come costruire le varie istruzioni select per estrarre le informazioni

Tabella persone

<u>idPersona</u>	nome	cognome	dataNasc	email	telefono	indirizzo	citta	stipendio
001	Mario	Rossi	01/06/1975	m.rossi@email.it	0001234	Via Roma, 123	Roma	794
002	Antonio	Verdi	15/09/1961	verdi@antonio.it	074532234	P.za S.giova 11	Pescara	928
003	Antonio	Bianchi	24/11/1978	bianchi@anto.it	06338293002	V.le dei giovani	Roma	825
004	Simona	Bsttiston	01/06/1967	s.b@email.it	045387658	P.za Fo	Milano	794
005	Daniela	Longo	29/12/1967	dany@longo.it	023435444	Via Roma, 85	Milano	1000
006	Giuseppe	Savo	13/05/1985	g@savo.it	063445466	Via dei portici, 32	Roma	1100

Query senza filtri:

SELECT <campi> FROM <tabella>

Questa è la versione più semplice di una select che, senza nessun filtro impostato sulla selezione, estrae tutti i campi presenti nella tabella specificata.

I campi che si vogliono estrarre si possono indicare separati da virgola.

Se si vuole estrarre tutti i campi della tabella si può utilizzare l'operatore asterisco * che ci permette, quindi, di non dover scrivere tutti i campi.

Esempi

Estrazione di tutti i campi della tabella “persone” senza filtri:

SELECT idPersona, nome, cognome, dataNasc, email, telefono, indirizzo, citta FROM persone;

*SELECT * FROM persone;*

Estrazione di alcuni i campi dalla tabella “persone”, senza filtri:

SELECT nome, cognome, email FROM persone

L'operatore AS:

L'istruzione SELECT estrae i campi specificati dalla tabella indicata, secondo gli eventuali parametri di ricerca.

Abbiamo detto che i dati verranno estratti in forma tabellare dove i nomi delle colonne saranno i nomi dei campi indicati, e le righe i record che soddisfano la ricerca.

E' anche possibile indicare dei nomi di colonna diversi da quelli iniziali. Tale operazione potrà risultare utile quando si lavorerà con più tabelle.

Si può quindi indicare un nome diverso da quello di origine per il campo estratto, attraverso l'operatore AS:

Esempi

Utilizzo dell'operatore AS

SELECT idPersona AS ID, nome, cognome, dataNasc AS data_nascita, email FROM persone;

L'operatore COUNT:

In una query è possibile utilizzare l'operatore count per avere come risultato della ricerca non i singoli valori trovati, ma il numero dei risultati.

Esempi

Utilizzo dell'operatore COUNT (Considerando la tabella di esempio i valori ritornati dal count per questo tipo di query sarà 6

```
SELECT COUNT(idPersona) AS Tot FROM persone;
```

```
SELECT COUNT(idPersona) AS Tot, idPersona AS ID, nome, cognome, dataNasc AS data_nascita,  
email FROM persone;
```

```
SELECT COUNT(*) FROM persone;
```


L'operatore SUM:

In una query è possibile utilizzare l'operatore sum per avere come risultato il totale calcolato sul campo specificato.

A differenza del del count non conteggia, ma somma i valori estratti di un campo numerico

Esempi

Utilizzo dell'operatore SUM (Considerando la tabella di esempio i valori ritornati dalla sommatoria per questo tipo di query sarà 5441

```
SELECT SUM(stipendio) as TotStip FROM persone;
```

L'operatore DISTINCT:

In una query è possibile utilizzare l'operatore distinct per filtrare i valori ripetuti, escludendo quindi i duplicati

Esempi

SELECT DISTINCT nome FROM persone;

Considerando la tabella di esempio i valori ritornati dal distinct saranno:

- *Mario*
- *Antonio*
- *Simona*
- *Daniela*
- *Giuseppe*

Query su più tabelle: le JOIN

```
SELECT [operatori] <elenco campi>  
FROM <tabella>  
[ JOIN <tabella> ON <campo di unione> ]
```

La JOIN è un'operazione che unisce il contenuto di una o più tabelle. E si può realizzare in tre modi diversi:

- Inner join
- Left outer join
- Right outer join

Di seguito analizzeremo i tre tipi di unione.

INNER JOIN

L'inner join, detto anche JOIN, è l'operazione di unione più utilizzata e conosciuta.

L'inner join unisce le due tabelle specificate, unendo i soli record che corrispondono al criterio specificato per la relazione; vengono quindi escluse tutte le righe che non soddisfano il criterio di relazione.

L'inner join viene rappresentato con questo formato:

```
SELECT [elenco campi]  
FROM tabella1 as t1 INNER JOIN tabella2 as t2  
ON t1.campo= t2.campo
```

Esempi:

Considerando le tabelle sotto riportate, scriviamo una query che estragga i dati dei clienti che abbiano almeno una fattura

Clienti

<u>idCliente</u>	nome	cognome	dataNasc
001	Mario	Rossi	01/06/1975
002	Antonio	Verdi	15/09/1961
003	Antonio	Bianchi	24/11/1978

Fatture

<u>idFattura</u>	<u>idCliente</u>	dataFattura	Importo
100	001	24/11/2011	128,77
101	002	04/08/2011	111,21
102	001	13/05/2011	89,77

```
SELECT c.nome, c.cognome  
FROM Clienti c INNER JOIN Fatture f
```

ON f.idCliente=f.idCliente

L'inner join si può anche scrivere in forma abbreviata, indicando la sola relazione senza utilizzare la parola chiave Inner join:

```
SELECT c.nome, c.cognome  
FROM Clienti c, Fatture f  
WHERE f.idCliente=f.idCliente
```

Nota: la clausola where, sulla quale ora non ci soffermiamo, verrà trattata ampiamente all'interno della lezione.

LEFT OUTER JOIN

Il left outer join, detto anche left join, relaziona due tabelle estraendo tutte le righe presenti nella tabella specificata a sinistra del join e riempiendo con NULL i campi il cui valore non esiste nella seconda tabella.

Le righe presenti unicamente nella tabella di destra non saranno affatto estratte.

Il left join viene rappresentato con questo formato:

```
SELECT [elenco campi]  
FROM tabella1 as t1 LEFT JOIN tabella2 as t2  
ON t1.campo= t2.campo
```

Esempi:

Considerando le tabelle sotto riportate, scriviamo una query che estragga tutti i dati dei clienti con i dati delle eventuali fatture

Clienti

<u>idCliente</u>	nome	cognome	dataNasc
001	Mario	Rossi	01/06/1975
002	Antonio	Verdi	15/09/1961
003	Antonio	Bianchi	24/11/1978

Fatture

<u>idFattura</u>	<u>idCliente</u>	dataFattura	Importo
100	001	24/11/2011	128,77
101	002	04/08/2011	111,21
102	111	04/08/2011	12,21

```
SELECT c.nome, c.cognome, f.Importo
FROM Clienti c LEFT JOIN Fatture f
ON f.idCliente=c.idCliente
```


La query di esempio estrarrà quindi i seguenti valori:

<i>nome</i>	<i>cognome</i>	<i>Importo</i>
<i>Mario</i>	<i>Rossi</i>	<i>128,77</i>
<i>Antonio</i>	<i>Verdi</i>	<i>111,21</i>
<i>Antonio</i>	<i>Bianchi</i>	

Si può notare come il cliente Antonio Bianchi venga comunque estratto anche senza avere una corrispondenza nella tabella delle fatture.

Non accade invece il contrario: la fattura numero 102 non viene estratta, perché non ha nessuna corrispondenza con la tabella dei clienti.

RIGHT OUTER JOIN

Il right outer join, detto anche right join, agisce in modo opposto al left join.

Estrae, quindi, tutte le righe presenti nella tabella specificata a destra del join e riempie con NULL i campi il cui valore non esiste nella prima tabella.

Le righe presenti unicamente nella tabella di sinistra non saranno affatto estratte.

Il right join viene rappresentato con questo formato:

```
SELECT [elenco campi]  
FROM tabella1 as t1 RIGHT JOIN tabella2 as t2  
ON t1.campo= t2.campo
```

Esempi:

Considerando le tabelle sotto riportate, scriviamo una query che estragga tutti i dati delle fatture con i dati degli eventuali clienti associati

Clienti

<u>idCliente</u>	nome	cognome	dataNasc
001	Mario	Rossi	01/06/1975
002	Antonio	Verdi	15/09/1961
003	Antonio	Bianchi	24/11/1978

Fatture

<u>idFattura</u>	<u>idCliente</u>	dataFattura	Importo
100	001	24/11/2011	128,77
101	002	04/08/2011	111,21
102	111	04/08/2011	12,21

```
SELECT f.idFattura, f.Importo, c.nome, c.cognome  
FROM Clienti c RIGHT JOIN Fatture f  
ON f.idCliente=f.idCliente
```

La query di esempio estrarrà quindi i seguenti valori:

<i><u>idFattura</u></i>	<i>Importo</i>	<i>nome</i>	<i>cognome</i>
100	128,77	Mario	Rossi
101	111,21	Antonio	Verdi
102	12,21		

Si può notare come la fattura numero 102 venga comunque estratta anche senza avere una corrispondenza nella tabella dei clienti.

Non accade invece il contrario: il cliente 003 non viene estratto, perché non ha nessuna corrispondenza con la tabella delle fatture.

La clausola WHERE

La clausola where è probabilmente la più utilizzata nelle query: questa infatti ha come scopo quello di limitare le selezioni, includendo nelle estrazioni i soli record i cui valori corrispondono ai criteri specificati.

La where può essere quindi considerata come un filtro da applicare alle nostre interrogazioni per estrarre i soli valori che ci interessano, e non tutti i record inseriti nelle tabelle.

La sua sintassi generale, all'interno di una query, è:

WHERE <campo> <predicato> <valori>

Per “campo” si intende un campo della tabella, mentre il “predicato” stabilisce la regola da verificare, e “valori” indica i valori indicati per verificare la regola.

Il predicato può assumere i seguenti valori:

=	<i>è uguale al valore specificato di seguito</i>
<	<i>è minore del valore specificato di seguito</i>
>	<i>è maggiore del valore specificato di seguito</i>
<=	<i>è minore o uguale al valore specificato di seguito</i>
>=	<i>è maggiore o uguale al valore specificato di seguito</i>
<>	<i>è differente dal valore specificato di seguito</i>
LIKE	<i>è come il valore specificato di seguito</i>
NOT LIKE	<i>non è come il valore specificato di seguito</i>
BETWEEN	<i>è compreso tra i due valori specificati di seguito</i>
IN	<i>corrisponde ad uno dei valori specificati di seguito</i>
NOT IN	<i>non corrisponde a nessuno dei valori specificati di seguito</i>
IS NULL	<i>è il valore NULL</i>

Note: *I test sui campi di tipo numerico si effettuano specificando valori numerici.*

I test sui campi di tipo stringa si effettuano specificando i valori racchiusi tra apici singoli.

Le ricerche sui campi testo, inoltre, non tengono conto delle differenze tra caratteri maiuscoli e minuscoli; si dice quindi che sono “case insensitive”.

Vediamo quindi con degli esempi, come costruire delle query contenenti la clausola where, e come questa si può utilizzare per ottenere i più svariati filtri da applicare ai dati memorizzati in un data base.

Prendiamo come esempio per le nostre query, la tabella di seguito riportata:

<i><u>idPersona</u></i>	<i>nome</i>	<i>cognome</i>	<i>annoNasc</i>	<i>email</i>	<i>indirizzo</i>	<i>citta</i>
1	Mario	Rossi	1975	m.rossi@email.it	Via Roma, 123	Roma
2	Antonio	Verdi	1961	verdi@antonio.it	P.za S.giovanni 11	Pescara
3	Antonio	Bianchi	1978	bianchi@anto.it	V.le dei giovani	Roma
4	Simona	Bsttiston	1967	s.b@email.it	P.za Fo	Milano
5	Daniela	Longo	1967	dany@longo.it	Via Roma, 85	Milano
6	Daniele	Savo	1985	d@savo.it	Via dei portici, 32	Roma

Nelle query a seguire vedremo come si costruisce la clausola e l'utilizzo dei predicati elencati in precedenza:

Estrarre tutte le persone con il campo citta uguale a roma

```
SELECT *  
FROM Persone p  
WHERE p.citta = 'Roma'
```

Questa query estrarrà le sole persone con id 1, 3 e 6 che sono le uniche a soddisfare i criteri di ricerca.

Estrarre tutte le persone con il campo citta uguale a messina

```
SELECT *  
FROM Persone p  
WHERE p.citta = 'messina'
```

Questa query non restituirà alcun record, in quanto non c'è nessuna persona che soddisfa i criteri di ricerca.

Estrarre tutte le persone con il campo citta uguale a roma

```
SELECT *  
FROM Persone p  
WHERE p.citta='Roma'
```

Questa query estrarrà le sole persone con id 001, 003 e 006 che sono le uniche a soddisfare i criteri di ricerca.

Estrarre tutte le persone con il campo idPersona minore a 3

```
SELECT *  
FROM Persone p  
WHERE p.idPersona < 3
```

Questa query estrarrà le sole persone con id 1 e 2.

Estrarre tutte le persone con il campo idPersona minore uguale a 3

```
SELECT *  
FROM Persone p  
WHERE p.idPersona <= 3
```

Questa query estrarrà le sole persone con id 1, 2 e 3.

Estrarre tutte le persone con il campo idPersona maggiore a 3

```
SELECT *  
FROM Persone p  
WHERE p.idPersona > 3
```

Estrarre tutte le persone con il campo idPersona maggiore uguale a 3

```
SELECT *  
FROM Persone p  
WHERE p.idPersona >= 3
```

Estrarre tutte le persone con il campo idPersona diverso da 1

```
SELECT *  
FROM Persone p  
WHERE p.idPersona <> 1
```

Questa query estrarrà tutte le persone ad esclusione di quella con id 1.

Prima di riportare degli esempi per il predicato like è necessario fare una piccola introduzione:

Il like serve per verificare che un campo contenga il valore specificato.

Con il like posso verificare se il campo deve contenere esattamente il valore specificato o una parte di esso. Questo si ottiene grazie a due simboli:

- Il simbolo _ che indica un solo carattere qualsiasi;
- Il simbolo % che indica qualsiasi combinazione di qualsiasi carattere.

Nota: il simbolo % si specifica ad inizio o fine stringa o in entrambe le posizioni.

Se indicato ad inizio stringa vuol dire che ci interessa che la stringa contenga solo gli ultimi caratteri specificati, senza dare importanza alle combinazioni dei caratteri precedenti:

nome like '%ia' → vuol dire tutti i nomi che finiscono con 'ia', come ad esempio Maria, Sofia, ecc.

Se indicato a fine stringa vuol dire che ci interessa esclusivamente come inizia il valore del campo:

nome like 'gian%' → vuol dire tutti i nomi che iniziano con 'gian', come ad esempio Gianni, Gianpiero, Gian Mario, ecc.

Se indicato ad inizio e fine stringa vuol dire che stiamo ricercando i campi che contengano la stringa indicata, ma senza preoccuparci di cosa contiene prima o dopo

email like '%anto%' → vuol dire tutte le email che contengono 'anto', come ad esempio verdi@antonio.it e bianchi@anto.it

Vediamo quindi degli esempi di utilizzo del like:

Estrarre tutte le persone con il campo nome che inizia per daniel e finisce un un carattere qualsiasi

```
SELECT *  
FROM Persone p  
WHERE p.nome like 'daniel_'
```

Restituirà i record 5 e 6 che hanno il campo nome con valore rispettivamente Daniela e Daniele. Se ci fosse stato ad esempio un Danielino, questo non sarebbe stato estratto in quanto il carattere _ considera un solo carattere.

Estrarre tutte le persone con il campo nome che finisce per “io”

```
SELECT *  
FROM Persone p  
WHERE p.nome like '%io'
```

Restituisce i record con id 1, 2 e 3 perché con nomi Mario, Antonio e Antonio

Estrarre tutte le persone con il campo email che contiene il valore ‘anto’

```
SELECT *  
FROM Persone p  
WHERE p.email like '%anto%'
```

Restituisce i record con id 2 e 3 perché con email verdi@antonio.it e bianchi@anto.it

Considerando che l'operatore not inverte ogni condizione, negandola, il predicato not like, si comporterà esattamente al contrario del like.

Estrarre tutte le persone con il campo nome che non finisce per "io"

```
SELECT *  
FROM Persone p  
WHERE p.nome not like '%io'
```

Restituisce i record della tabella ad esclusione di quelli con id 1, 2 e 3 perché con nomi Mario, Antonio e Antonio

Il predicato ***between*** viene utilizzato per mettere nelle condizioni un range di valori.

Serve quindi per estrarre i record con il campo avente un valore compreso tra quelli specificati.

Estrarre tutte le persone con il campo anno di nascita compreso tra il 1960 e il 1970

```
SELECT *  
FROM Persone p  
WHERE p.dataNasc between 1960 and 1970
```

Restituisce i record con id 2, 4 e 5 perché con date di nascite comprese nell'intervallo specificato.

Il predicato ***in*** offre la possibilità di includere nella selezione, una lista di valori per il campo specificato.

Una query con predicato *in*, quindi, prenderà in considerazione tutti i campi che hanno esclusivamente come valore quelli indicati nella query. Vediamo un esempio:

Estrarre tutte le persone con il campo anno di nascita uguale ad uno dei valori specificati: 1963, 1965, 1967, 1969

```
SELECT *  
FROM Persone p  
WHERE p.annoNasc in (1963, 1965, 1967, 1969)
```

Restituisce i record con id 4 e 5 perché con anno di nascita uguale ad uno dei quattro specificati

Il predicato ***not in*** è la negazione dell' ***in***, che quindi si comporta in modo opposto al predicato appena visto.

Il ***not in***, quindi, prenderà in considerazione tutti i campi che non hanno come valore quelli indicati. Vediamo un esempio:

Estrarre tutte le persone con il campo anno di nascita diverso dai valori specificati: 1963, 1965, 1967, 1969

```
SELECT *  
FROM Persone p  
WHERE p.annoNasc not in (1963, 1965, 1967, 1969)
```

Restituisce tutti i record ed esclusione di quelli con id 4 e 5 perché con anno di nascita uguale ad uno dei quattro specificati.

Il predicato ***is null*** viene utilizzato per restituire i soli record il cui campo al quale viene applicato, abbia valore null. Il predicato ***is not null*** è invece il suo inverso. Esempi:

Estrarre tutte le persone con il campo anno di nascita uguale a null

```
SELECT *  
FROM Persone p  
WHERE p.annoNasc is null
```

Non restituisce record perché nella tabella utilizzata per l'esempio nessun campo anno di nascita ha valore null.

Estrarre tutte le persone con il campo anno di nascita diverso da null

```
SELECT *  
FROM Persone p  
WHERE p.annoNasc is not null
```

Restituisce tutti i record della tabella perchè nessun campo anno di nascita ha valore null.

Nelle query è possibile combinare tra loro più predicati, questo per rendere l'interrogazione più precisa e dettagliata.

E' possibile legare più condizioni tra loro, nella clausola where, utilizzando gli operatori booleani **and** e **or** dove:

- **and** specifica che il criterio precedente e quello successivo debbano restituire risultato vero
- **or** specifica che almeno uno dei due criteri debba restituire risultato vero

Vediamo degli esempi.

Estrarre tutte le persone con il campo nome che finisce per “io”, con anno di nascita diverso da quelli indicati

```
SELECT *  
FROM Persone p  
WHERE p.nome like '%io' and p.annoNasc not in (1975, 1978)
```

Restituisce solo il record con id 2 perché l'unico con nome che termina per 'io' e con anno di nascita diverso dal 1975 e 1978

Estrarre tutte le persone con il campo nome che finisce per “io”, oppure con anno di nascita uguale

```
SELECT *  
FROM Persone p  
WHERE p.cognome like '%di' or p.email like '%anto%'
```

Restituisce i record con id 2 e 3 perché nonostante solo quello con id 2 termini con 'di', l'operatore or determina un'altra estrazione

La clausola GROUP BY

La clausola **group by** viene utilizzata per raggruppare i risultati di una select in base al campo indicato.

Il **group by** si può utilizzare solo insieme ad un operatore di conteggio come ad esempio l'operatore count o sum.

Di seguito alcuni esempi partendo dalla seguente tabella:

<u>idPersona</u>	nome	cognome	annoNasc	email	indirizzo	citta
1	Mario	Rossi	1975	m.rossi@email.it	Via Roma, 123	Roma
2	Antonio	Verdi	1961	verdi@antonio.it	P.za S.giovanni 11	Pescara
3	Antonio	Bianchi	1978	bianchi@anto.it	V.le dei giovani	Roma
4	Simona	Bsttiston	1967	s.b@email.it	P.za Fo	Milano
5	Daniela	Longo	1967	dany@longo.it	Via Roma, 85	Milano
6	Daniele	Savo	1985	d@savo.it	Via dei portici, 32	Roma

Contare le persone raggruppandole per città

```
SELECT città, count(*) as tot  
FROM Persone p  
GROUP BY città
```

Restituisce i seguenti valori:

<i>città</i>	<i>tot</i>
<i>Roma</i>	<i>3</i>
<i>Pescara</i>	<i>1</i>
<i>Milano</i>	<i>2</i>

*Insieme alla group by si può utilizzare anche un vincolo sui dati risultati per filtrarli, come avviene normalmente con la where. Con questa clausola, per filtrare i risultati si utilizza la **having** che vediamo di seguito.*

La clausola **HAVING**

La clausola ***having*** viene utilizzata per filtrare i risultati delle operazioni di raggruppamento, e si può paragonare ad una **where**, anche se il suo utilizzo è limitato alle aggregazioni.

La **having** può operare sia sui campi utilizzati per le funzioni di aggregazione (**sum**, **count**) che su quelli specificati nella clausola **group by**.

Vediamo un esempio per chiarirne l'utilizzo, basandoci sempre sulla tabella riportata per la **group by**.

Contare le persone raggruppandole per città dove per ogni raggruppamento ci siano almeno 2 città

```
SELECT citta, count(citta) as tot  
FROM Persone p  
GROUP BY citta  
HAVING count(citta) >1
```

Restituisce i seguenti valori:

<i>citta</i>	<i>tot</i>
<i>Roma</i>	<i>3</i>
<i>Milano</i>	<i>2</i>

Notare come, a differenza della query precedente, il totale per la città di Pescara non viene estratto, in quanto il suo totale è inferiore a 2, proprio come richiesto dalla condizione indicata nella clausola having.

La clausola **ORDER BY**

La clausola ***order by*** viene utilizzata per ordinare i risultati delle operazioni di estrazione.

La sua sintassi è:

ORDER BY <campo> [*ASC* , *DESC*]

E' possibile ordinare uno o più campi in ordine crescente, cioè dalla A alla Z (*asc*), o decrescente (*desc*), cioè dalla Z alla A.

L'ordinamento ascendente è quello di default e si può quindi omettere dopo il campo. Per ordinare invece in modo decrescente, è necessario specificare *desc* subito dopo il nome del campo sul quale si applica l'ordinamento.

Prendiamo per gli esempi la seguente tabella:

<u>idPersona</u>	nome	cognome	annoNasc	email	indirizzo	citta
1	Mario	Rossi	1975	m.rossi@email.it	Via Roma, 123	Roma
2	Antonio	Verdi	1961	verdi@antonio.it	P.za S.giovanni 11	Pescara
3	Antonio	Bianchi	1978	bianchi@anto.it	V.le dei giovani	Roma
4	Simona	Bsttiston	1967	s.b@email.it	P.za Fo	Milano
5	Daniela	Longo	1967	dany@longo.it	Via Roma, 85	Milano
6	Daniele	Savo	1985	d@savo.it	Via dei portici, 32	Roma

Estrarre tutti i record della tabella ordinati in modo crescente per nome

```
SELECT *  
FROM Persone p  
ORDER BY nome
```

Estrarre tutti i record della tabella ordinati in modo crescente per nome e cognome

```
SELECT *  
FROM Persone p  
ORDER BY nome, cognome
```

Estrarre tutti i record della tabella ordinati in modo decrescente per cognome

```
SELECT nome, cognome  
FROM Persone p  
ORDER BY cognome desc
```

Estrarre tutti i record della tabella ordinati in modo crescente per nome e cognome

```
SELECT nome, cognome, email  
FROM Persone p  
WHERE citta like 'roma'  
ORDER BY nome asc
```

Query di aggiornamento

Come anticipato, nel DML (*Data Manipulation Language*) rientrano le query così dette di estrazione, appena viste attraverso l'istruzione `select`, e quelle dette di aggiornamento.

Questo tipo di query ci permettono di eseguire sul nostro data base operazioni di inserimento, modifica o cancellazione dei dati e sono identificate con le istruzioni `INSERT INTO`, `UPDATE` e `DELETE`.

Vediamo quindi nello specifico tali istruzioni.



L'istruzione INSERT INTO

L'istruzione **insert into**

L'istruzione ***insert into***, in sql, ha come scopo quello di inserire nuovi record all'interno di una tabella già esistente. Per creare una nuova tabella far riferimento all'istruzione create table vista del DDL (Data Definition Language). La sintassi dell'istruzione ***insert into*** è:

```
INSERT INTO <tabella> [ (campi) ] VALUES( <valori> )
```

Vediamo nello specifico la struttura del comando.

L'istruzione ***insert into*** viene seguita dal nome della tabella su cui eseguire gli inserimenti; poi, facoltativo e dentro le parentesi tonde troviamo il nome dei campi per i quali vogliamo inserire i valori.

A seguire viene specificato l'operatore ***values***, il quale conterrà, specificati tra parentesi tonde, i valori da inserire per i campi; tali valori dovranno essere specificati tra virgolette, se fanno riferimento a campi di tipo testo e senza se relativi invece a campi numerici.

La specifica dei campi, dopo l'istruzione ***insert into***, diventa necessaria se vogliamo inserire non tutti i campi definiti per la tabella, ma solo alcuni.

Prendiamo per gli esempi la seguente tabella:

<u>idPersona</u>	nome	cognome	annoNasc	email	indirizzo	citta
1	Mario	Rossi	1975	m.rossi@email.it	Via Roma, 123	Roma
2	Antonio	Verdi	1961	verdi@antonio.it	P.za S.giovanni 11	Pescara
3	Antonio	Bianchi	1978	bianchi@anto.it	V.le dei giovani	Roma
4	Simona	Bsttiston	1967	s.b@email.it	P.za Fo	Milano
5	Daniela	Longo	1967	dany@longo.it	Via Roma, 85	Milano
6	Daniele	Savo	1985	d@savo.it	Via dei portici, 32	Roma

Inserire un nuovo record nella tabella, che preveda l'indicazione di tutti i campi definiti

```
INSERT INTO Persone (idPersona, nome, cognome, annoNasc, email, indirizzo, citta)  
VALUES (7, "Luigi", "Antonelli", 1980, "l.a@email.it", "via roma, 22", "Pisa")
```

Inserendo un valore per ciascun campo, l'elenco dei campi può essere omissso, come nell'esempio che segue

Inserire un nuovo record nella tabella, con un valore indicato per ciascun campo

```
INSERT INTO Persone  
VALUES (7, "Luigi", "Antonelli", 1980, "l.a@email.it", "via roma, 22", "Pisa")
```

Inserire un nuovo record nella tabella, con dei valori non definiti per alcuni campi

```
INSERT INTO Persone (idPersona, nome, cognome, annoNasc, email, indirizzo, citta)  
VALUES (7, "Luigi", "Antonelli", 1980, "l.a@email.it", "", "")
```

Inserire un nuovo record nella tabella, che preveda l'inserimento di soli alcuni campi

```
INSERT INTO Persone (nome, cognome, email)  
VALUES ("Luigi", "Antonelli", "l.a@email.it")
```

```
INSERT INTO Persone (nome, cognome, indirizzo, citta)  
VALUES ("Luigi", "Antonelli", "via roma, 22", "Pisa")
```

L'istruzione UPDATE

L'istruzione **update**

L'istruzione ***update***, nell'sql, ha come scopo quello di modificare i valori dei campi di uno o più record, precedentemente inseriti nella tabella specificata. La sintassi dell'istruzione ***update*** è:

```
UPDATE <tabella> SET <campo>=<valore>  
[ WHERE <condizione> ]
```

Vediamo nello specifico la struttura del comando.

L'istruzione ***update*** viene seguita dal nome della tabella su cui eseguire l'aggiornamento; viene quindi specificato l'operatore ***set***, al quale segue il nome del campo e dopo l'uguale il valore da sostituire al precedente; questo valore dovrà essere specificato tra virgolette, se si fa riferimento ad un campo di tipo testo, e senza per un campo di tipo numerico.

L'istruzione ***update*** può essere seguita dal costrutto ***where***, che ne determina il solo o i soli record sui quali effettuare l'aggiornamento.

In assenza del filtro ottenuto dalla ***where***, infatti, si modificherebbero indistintamente tutti i record presenti in tabella.

Prendiamo per gli esempi la seguente tabella:

<u>idPersona</u>	nome	cognome	annoNasc	email	indirizzo	citta
1	Mario	Rossi	1975	m.rossi@email.it	Via Roma, 123	Roma
2	Antonio	Verdi	1961	verdi@antonio.it	P.za S.giovanni 11	Pescara
3	Antonio	Bianchi	1978	bianchi@anto.it	V.le dei giovani	Roma
4	Simona	Bsttiston	1967	s.b@email.it	P.za Fo	Milano
5	Daniela	Longo	1967	dany@longo.it	Via Roma, 85	Milano
6	Daniele	Savo	1985	d@savo.it	Via dei portici, 32	Roma

Modificare il campo città di tutti i record della tabella

```
UPDATE Persone SET citta = "Roma"
```

L'esecuzione di questa query, mi cambierebbe il valore del campo città a tutti i record; se non voglio sostituire tutti i valori indistintamente, dovrò applicare un filtro utilizzando il costrutto where.

Modificare il campo città ai record con idPersona = 5

```
UPDATE Persone SET città = "Milano"  
WHERE idPersona = 5
```

Applicando la where, ottengo come risultato quello di modificare un solo record. Con la where si possono modificare anche più record alla volta.

Modificare il campo città ai record con idPersona = 5 e 4

```
UPDATE Persone SET città = "Milano"  
WHERE idPersona IN(4, 5)
```

Applicando questo filtro, ottengo come risultato quello di modificare due record, quelli con id 4 e 5.

Con l'istruzione ***update*** è possibile modificare anche più campi contemporaneamente, basta indicare tutti i campi seguiti dal nuovo valore e separati da virgola, dopo l'operatore ***set***.

Modificare il campo città e indirizzo ai record con idPersona = 5 e 4

```
UPDATE Persone SET indirizzo = "piazza della liberazione, 11", città = "Genova"  
WHERE idPersona IN(4, 5)
```

Modificare il campo città e indirizzo ai record con nome Antonio o con nome che inizia per Daniel

```
UPDATE Persone SET indirizzo = "", città = "Roma"  
WHERE nome = "antonio" OR nome like "daniel%"
```

Con questa query modifico i campi delle persone con id 2, 3, 5 e 6.

L'istruzione DELETE

L'istruzione delete

L'istruzione ***delete***, in sql, ha come scopo quello di eliminare i record specificati.

La sintassi dell'istruzione ***delete*** è:

```
DELETE FROM <tabella>  
[ WHERE <condizione> ]
```

Vediamo nello specifico la struttura del comando.

L'istruzione ***delete*** viene seguita dal nome della tabella su cui eseguire la cancellazione.

L'istruzione ***delete*** può essere seguita dal costrutto ***where***, che ne determina il solo o i soli record sui quali effettuare l'aggiornamento.

In assenza del filtro ottenuto dalla ***where***, infatti, si eliminerebbero indistintamente tutti i record presenti in tabella.

Prendiamo per gli esempi la seguente tabella:

<u>idPersona</u>	nome	cognome	annoNasc	email	indirizzo	citta
1	Mario	Rossi	1975	m.rossi@email.it	Via Roma, 123	Roma
2	Antonio	Verdi	1961	verdi@antonio.it	P.za S.giovanni 11	Pescara
3	Antonio	Bianchi	1978	bianchi@anto.it	V.le dei giovani	Roma
4	Simona	Bsttiston	1967	s.b@email.it	P.za Fo	Milano
5	Daniela	Longo	1967	dany@longo.it	Via Roma, 85	Milano
6	Daniele	Savo	1985	d@savo.it	Via dei portici, 32	Roma

Eliminare tutti i record della tabella

DELETE FROM Persone

L'esecuzione di questa query, elimina tutti i record presenti. Se non voglio cancellare tutti i valori indistintamente, dovrò applicare un filtro utilizzando il costrutto where.

Eliminare il record con idPersona = 5

```
DELETE FROM Persone  
WHERE idPersona = 5
```

Applicando la where, ottengo come risultato quello di eliminare un solo record. Con la where si possono cancellare anche più record alla volta.

Eliminare i record con idPersona = 5 e 4

```
DELETE FROM Persone  
WHERE idPersona IN(4, 5)
```

Applicando questo filtro, ottengo come risultato quello di eliminare due record, quelli con id 4 e 5.

Eliminare i record delle persone con nome Antonio o con nome che inizia per Daniel

```
DELETE FROM Persone  
WHERE nome = "antonio" OR nome like "daniel%"
```

